Reforge Audit Report

legacy-system-repo · Pydantic 1.10.12 -> 2.14.1 · 2026-10-28 07:35 UTC

PARITY PASS	Original and refactored code behave the same: tests, return values, model schemas and entrypoint output match.

1/1 Vulnerabilities resolved 0 open in 2.14.1	**100% -> 100%** Test pass rate 6/6 tests after refactor	**100%** Output parity 18 identical, 0 equivalent of 18
5.28x Hot-path speedup test_validation_speed_benchmark	**$0.1219** AI spend this run 27 dispatches, 138,454 tokens	**$0.2016** Saved vs Ultra-only 62% cheaper

Summary

- **Migration.** 3 file(s) refactored (+16 / -18 lines) from a 12-step plan; Stage 4 status: **verified**, with escalation.
- **Security.** 1 known advisory(ies) affected pydantic 1.10.12; 1 resolved by moving to 2.14.1 (re-checked on osv.dev for pydantic 2.14.1).
- **Behaviour.** 6/6 tests passed on the original (v1), 6/6 on the refactor (v2); 0 regression(s). 18 recorded outputs compared, 0 nondeterministic (judged on structure).
- **Performance.** Test suite 1.17x faster; peak memory 63.4 -> 60.4 MB.
- **Cost.** $0.1219 spent across 3 model(s); the same tokens on Nemotron Ultra alone would cost $0.3235.
- **Grounding.** 9 official documentation page(s) cited for 10 migrated symbol(s).

1. Security vulnerabilities resolved

Advisories from OSV.dev for **pydantic 1.10.12** (original pin), checked against **2.14.1** (post-refactor pin). Re-checked on OSV.dev for pydantic 2.14.1. Advisory ids that OSV lists for the same issue are grouped in one row.

Advisory	Description	In 1.10.12	In 2.14.1
GHSA-mr82-8j83-vxmv PYSEC-2026-1812 (aliases)	Regular expression denial of service in Pydantic < 2.4.0, < 1.10.13 allows remote attackers to cause denial of service via a crafted email string.	**Affected**	**Resolved**

2. Official documentation used

Every change was grounded in the Pydantic v2 docs before patching. Pages below were retrieved live (Tavily, restricted to docs.pydantic.dev) or come from Reforge's curated migration KB.

Source	Symbols grounded	Plan steps	Origin
https://docs.pydantic.dev/migration	.dict(), .json(), .parse_obj(), @root_validator, @validator, Field(min_items=...), Field(regex=...), anystr_strip_whitespace, class Config	-	live
https://docs.pydantic.dev/latest/migration/#changes-to-pydanticbasemodel	.dict(), .json(), .parse_obj()	10, 11, 12	KB
https://docs.pydantic.dev/latest/migration/#changes-to-validators	@root_validator, @validator	2, 7, 8	KB
https://docs.pydantic.dev/latest/migration/#changes-to-config	anystr_strip_whitespace, class Config	4, 6	KB
https://docs.pydantic.dev/latest/migration/#changes-to-pydanticfield	Field(min_items=...), Field(regex=...)	3, 5	KB
https://docs.pydantic.dev/latest/migration/#changes-to-json-schema-generation	.schema()	9	KB
https://docs.pydantic.dev/latest/migration/#install-pydantic-v2	-	1	KB
https://docs.pydantic.dev/2.0/usage/model_config	-	-	live
https://docs.pydantic.dev/2.12/concepts/config	-	-	live

Symbol mapping (v1 -> v2)

v1 symbol	v2 replacement	Topic	Evidence
`Field(regex=...)`	`Field(pattern=...)`	field	regex (use pattern instead)
`@root_validator`	`@model_validator`	validators	@root_validator has been deprecated, and should be replaced with @model_validator, which also provides new features and improvements. Be aware that the [...]
`@validator`	`@field_validator`	validators	@validator has been deprecated, and should be replaced with @field_validator, which provides various new features and improvements.
`anystr_strip_whitespace`	`str_strip_whitespace`	config	anystr_strip_whitespace -> str_strip_whitespace
`class Config`	`model_config = ConfigDict(...)`	config	In Pydantic V2, to specify config on a model, you should set a class attribute called model_config to be a dict with the key/value pairs you want to be [...]
`.dict()`	`model_dump`	model_methods	dict() -> model_dump()
`.json()`	`model_dump_json`	model_methods	json() -> model_dump_json()
`.parse_obj()`	`model_validate`	model_methods	parse_obj() -> model_validate()
`.schema()`	`model_json_schema`	model_methods	docs: https://docs.pydantic.dev/latest/migration/#changes-to-json-schema-generation (KB)

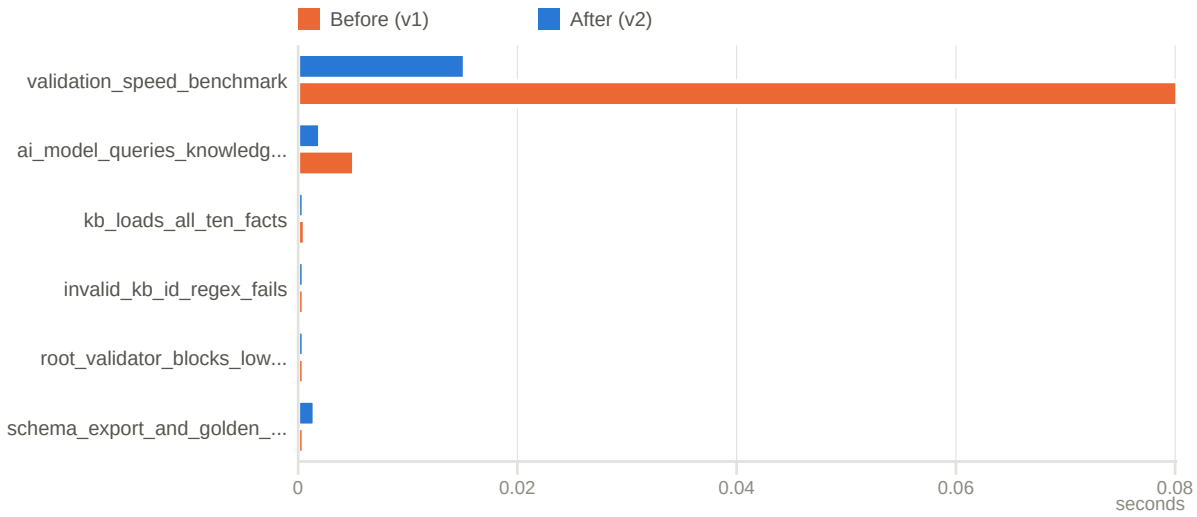
v1 symbol	v2 replacement	Topic	Evidence
`Field(min_items=...)`	`Field(min_length=...)`	field	min_items (use min_length instead)

3. Performance matrix

Same tests, same machine, original code on Pydantic v1 vs refactored code on Pydantic v2 (median of 3 timed runs each, network blocked, secrets blanked). For reference, the original code under v2 without the refactor passed 0/6 tests (Stage 2: collection failed: src/test.py (6 tests blocked)).

Metric	Before (v1 1.10.12, original)	After (v2 2.14.1, refactored)	Change
Test pass rate	100.0%	100.0%	+0.0 pts
Tests passed	6/6	6/6	0 regressed, 0 fixed
Output parity (return values + stdout)	reference	100.0% match	0 similar, 0 divergent
Model schema parity	2 models	2 identical	0 drift, 0 changed
Peak memory (test run)	63.4 MB	60.4 MB	-3.0 MB
Import footprint	51.2 MB	51.9 MB	+0.6 MB
Test suite time (median)	1.020 s	0.874 s	1.17x faster
Process wall time (median)	1.562 s	1.388 s	1.13x faster
Hot path: test_validation_speed_benchmark	0.0802 s	0.0152 s	5.28x faster

Per-test latency



Output parity detail

Return values of the repo's top-level functions and each test's stdout were recorded on both sides (2 runs each). **identical** = same value; **equivalent** = same structure and types, different values (expected for LLM text, hashes, timestamps); **similar** = structure overlaps >= 80%; **divergent** = structure changed.

Function	Calls	Identical	Equivalent	Similar	Divergent
`src/model.py::get_system_prompt_schema`	3	3	0	0	0
`src/model.py::golden_master_hash`	1	1	0	0	0
`src/model.py::load_kb`	6	6	0	0	0
`src/model.py::query_ai_model`	1	1	0	0	0
`src/model.py::validate_ai_response`	7	7	0	0	0

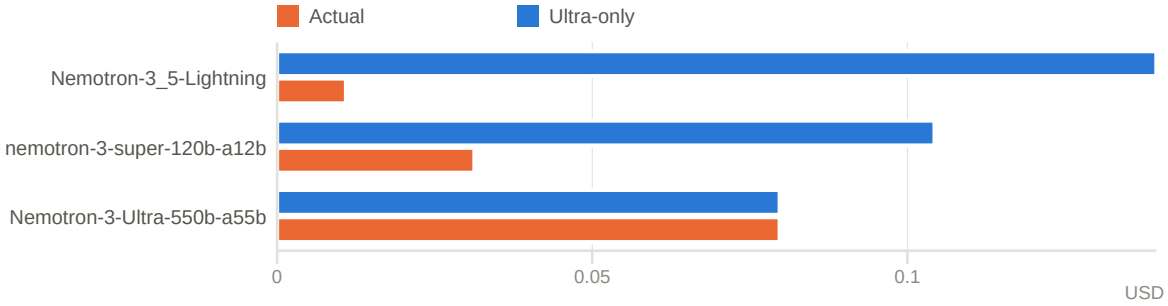
4. AI cost ledger

Every Nemotron dispatch of this run (Stage 3 planning + Stage 4 patching/healing), priced with data/model_catalog.md. The counterfactual prices the same tokens on Nemotron 3 Ultra, the model a single-model agent would use; cached replies cost $0.

Model	Tier	Stages	Calls (cached)	Input tok	Output tok	Spend	Ultra-only	Saved
`Nemotron-3_5-Lightning`	lightning	patch	11 (0)	12,649	42,281	$0.0109	$0.1395	$0.1286
`nemotron-3-super-120b-a12b`	super	heal, patch	8 (0)	13,336	30,319	$0.0313	$0.1043	$0.0730
`Nemotron-3-Ultra-550b-a55b`	ultra	heal, patch, plan	8 (0)	19,944	19,925	$0.0797	$0.0797	$0.0000
Total			27 (0)	45,929	92,525	**$0.1219**	$0.3235	**$0.2016**

Ledger line	Amount
Actual spend (live calls)	$0.121913
Ultra-only counterfactual (all tokens, no cache)	$0.323504
Saved by routing (Lightning / Super first, Ultra on escalation)	$0.201591
Saved by reply cache	$0.000000
Total money saved	**$0.201591 (62.3%)**
First-pass success	6/27 dispatches

Spend per model vs Ultra-only



Routing log (Stage 4)

11/12 plan steps applied; final verification 6/6 passed. Each step starts on the cheapest model and escalates only when its patch does not apply.

Step	File / action	Route	Result
1	`requirements.txt` bump pydantic to ==2.14.1	plan	Hunk applied cleanly
2	`src/schema.py` Update imports: add ConfigDict, field_validator, model_validator; drop validator, root_validator	lightning -> super	6 hunks applied (whitespace-tolerant match)
3	`src/schema.py` Rename Field regex parameter to pattern	lightning -> super -> ultra -> plan	No change needed (already applied by an earlier step)
4	`src/schema.py` Replace nested class Config with model_config = ConfigDict(...) for SourceMatch	lightning	2 hunks applied (whitespace-tolerant match)
5	`src/schema.py` Rename Field min_items parameter to min_length	lightning	2 hunks applied (whitespace-tolerant match)

Step	File / action	Route	Result
6	`src/schema.py` Replace nested class Config with model_config = ConfigDict(...) for AIQueryResult	lightning -> super -> ultra	Hunk applied (whitespace-tolerant match)
7	`src/schema.py` Replace @validator(pre=True) with @field_validator(mode='before') + @classmethod for SourceMatch	lightning -> super -> ultra	Hunk applied (whitespace-tolerant match)
8	`src/schema.py` Replace @root_validator with @model_validator(mode='after') for AIQueryResult	lightning -> super -> ultra	Hunk applied (whitespace-tolerant match)
9	`src/model.py` Replace .schema() with .model_json_schema() preserving v1 ref format	lightning	3 hunks applied (whitespace-tolerant match)
10	`src/model.py` Replace .parse_obj() with .model_validate()	lightning	2 hunks applied (whitespace-tolerant match)
11	`src/model.py` Replace .dict() with .model_dump()	lightning -> super -> ultra	Hunk applied (whitespace-tolerant match)
12	`src/model.py` Replace .json(sort_keys=True) with json.dumps(model_dump(mode='json'), sort_keys=True)	lightning -> super -> ultra	Hunk applied (whitespace-tolerant match)
heal	Test-driven fix after first verify (1/6 passed)	super	SEARCH block not found in file
heal	Test-driven fix after first verify (1/6 passed)	ultra	Hunk applied (whitespace-tolerant match)

All-time (data/telemetry.jsonl)

43 live dispatches, 163,660 tokens: spent $0.1542 vs $0.3502 Ultra-only, saved $0.1960 (56.0%).

5. Code diffs

3 refactored file(s). Original (left / -) vs refactored (right / +). Full patches: reforge.patch and diffs/ next to this PDF.

`requirements.txt` +1 -1

Side by side

1	pydantic==1.10.12	1	pydantic==2.14.1
2	openai>=1.50.0	2	openai>=1.50.0
3	pytest==8.3.3	3	pytest==8.3.3
4	python-dotenv	4	python-dotenv

Unified (git)

```
diff --git a/requirements.txt b/requirements.txt
--- a/requirements.txt
+++ b/requirements.txt
@@ -1,4 +1,4 @@
-pydantic==1.10.12
+pydantic==2.14.1
 openai>=1.50.0
 pytest==8.3.3
 python-dotenv
\ No newline at end of file
```

`src/model.py` +4 -4

Side by side

	... 16 unchanged lines ...		
17	def get_system_prompt_schema() -> Dict[str, Any]:	17	def get_system_prompt_schema() -> Dict[str, Any]:
18	"""Generates the JSON schema injected into the AI system prompt."""	18	"""Generates the JSON schema injected into the AI system prompt."""
19	# Breaks in v2: .schema() -> use .model_json_schema()	19	# Breaks in v2: .schema() -> use .model_json_schema()
20	s = AIQueryResult.schema()	20	s = AIQueryResult.model_json_schema(ref_template="#/definitions/{model}")
21	return {	21	return {
22	"title": s["title"],	22	"title": s["title"],
23	"required": sorted(s.get("required", [])),	23	"required": sorted(s.get("required", [])),
	... 64 unchanged lines ...		
88	"""Validates raw AI JSON output against AIQueryResult and serializes it."""	88	"""Validates raw AI JSON output against AIQueryResult and serializes it."""
89	kb = kb or load_kb()	89	kb = kb or load_kb()
90	# Breaks in v2: .parse_obj() -> use .model_validate()	90	# Breaks in v2: .parse_obj() -> use .model_validate()
91	validated = AIQueryResult.parse_obj(raw_payload)	91	validated = AIQueryResult.model_validate(raw_payload)
92		92	
93	for src in validated.sources:	93	for src in validated.sources:
94	if src.kb_id not in kb:	94	if src.kb_id not in kb:
95	raise ValueError(f"Unknown KB ID: {src.kb_id}")	95	raise ValueError(f"Unknown KB ID: {src.kb_id}")
96		96	
97	# Breaks in v2: .dict() & .json() -> use .model_dump() & .model_dump_json()	97	# Breaks in v2: .dict() & .json() -> use .model_dump() & .model_dump_json()
98	return {	98	return {
99	"structured": validated.dict(),	99	"structured": validated.model_dump(),
100	"serialized": validated.json(sort_keys=True),	100	"serialized": json.dumps(validated.model_dump(mode='json'), sort_keys=True),
101	}	101	}
102		102	

103	103
... 13 unchanged lines ...	

Unified (git)

```
diff --git a/src/model.py b/src/model.py
--- a/src/model.py
+++ b/src/model.py
@@ -17,7 +17,7 @@
 def get_system_prompt_schema() -> Dict[str, Any]:
     """Generates the JSON schema injected into the AI system prompt."""
     # Breaks in v2: .schema() -> use .model_json_schema()
-    s = AIQueryResult.schema()
+    s = AIQueryResult.model_json_schema(ref_template="#/definitions/{model}")
     return {
         "title": s["title"],
         "required": sorted(s.get("required", [])),
@@ -88,7 +88,7 @@
     """Validates raw AI JSON output against AIQueryResult and serializes it."""
     kb = kb or load_kb()
     # Breaks in v2: .parse_obj() -> use .model_validate()
-    validated = AIQueryResult.parse_obj(raw_payload)
+    validated = AIQueryResult.model_validate(raw_payload)

     for src in validated.sources:
         if src.kb_id not in kb:
@@ -96,8 +96,8 @@

     # Breaks in v2: .dict() & .json() -> use .model_dump() & .model_dump_json()
     return {
-        "structured": validated.dict(),
-        "serialized": validated.json(sort_keys=True),
+        "structured": validated.model_dump(),
+        "serialized": json.dumps(validated.model_dump(mode='json'), sort_keys=True),
     }
```

src/schema.py +11 -13

Side by side

```
from typing import List
from pydantic import BaseModel, Field, validator, root_validator


class SourceMatch(BaseModel):
    # Breaks in v2: 'regex' removed -> use 'pattern'
    kb_id: str = Field(..., regex=r"^KB-\d{2}$")
    fact: str

    # Breaks in v2: 'class Config' -> use model_config = ConfigDict(...)
    class Config:
        anystr_strip_whitespace = True
        extra = "forbid"


class AIQueryResult(BaseModel):
    query: str
    answer: str
    confidence: float = Field(..., ge=0.0, le=1.0)
    # Breaks in v2: 'min_items' removed -> use 'min_length'
    sources: List[SourceMatch] = Field(..., min_items=1)
    verified: bool = True

    class Config:
        anystr_strip_whitespace = True

    # Breaks in v2: @validator(pre=True) -> use @field_validator(mode="before") + @classmethod
    @validator("query", pre=True)

    def ensure_question(cls, v: str) -> str:
        if not isinstance(v, str) or not v.strip().endswith("?"):
            raise ValueError("Query must end with '?'")
        return v.strip()

    # Breaks in v2: @root_validator -> use @model_validator(mode="after")
    @root_validator
    def validate_verification_threshold(cls, values: dict) -> dict:
        if values.get("confidence", 0.0) < 0.6 and values.get("verified", False):
            raise ValueError("Low-confidence answers cannot be marked verified")
        return values
```

```
from typing import List
from pydantic import BaseModel, Field, ConfigDict, field_validator, model_validator


class SourceMatch(BaseModel):
    # Breaks in v2: 'regex' removed -> use 'pattern'
    kb_id: str = Field(..., pattern=r"^KB-\d{2}$")
    fact: str

    # Breaks in v2: 'class Config' -> use model_config = ConfigDict(...)
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")


class AIQueryResult(BaseModel):
    query: str
    answer: str
    confidence: float = Field(..., ge=0.0, le=1.0)
    # Breaks in v2: 'min_items' removed -> use 'min_length'
    sources: List[SourceMatch] = Field(..., min_length=1)
    verified: bool = True

    model_config = ConfigDict(str_strip_whitespace=True)

    # Breaks in v2: @validator(pre=True) -> use @field_validator(mode="before") + @classmethod
    @field_validator("query", mode="before")
    @classmethod
    def ensure_question(cls, v: str) -> str:
        if not isinstance(v, str) or not v.strip().endswith("?"):
            raise ValueError("Query must end with '?'")
        return v.strip()

    # Breaks in v2: @root_validator -> use @model_validator(mode="after")
    @model_validator(mode="after")
    def validate_verification_threshold(self) -> "AIQueryResult":
        if self.confidence < 0.6 and self.verified:
            raise ValueError("Low-confidence answers cannot be marked verified")
        return self
```

Unified (git)

```
diff --git a/src/schema.py b/src/schema.py
--- a/src/schema.py
+++ b/src/schema.py
@@ -1,16 +1,14 @@
 from typing import List
-from pydantic import BaseModel, Field, validator, root_validator
+from pydantic import BaseModel, Field, ConfigDict, field_validator, model_validator


 class SourceMatch(BaseModel):
     # Breaks in v2: 'regex' removed -> use 'pattern'
-    kb_id: str = Field(..., regex=r"^KB-\d{2}$")
+    kb_id: str = Field(..., pattern=r"^KB-\d{2}$")
     fact: str

     # Breaks in v2: 'class Config' -> use model_config = ConfigDict(...)
-    class Config:
-        anystr_strip_whitespace = True
-        extra = "forbid"
+    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")


 class AIQueryResult(BaseModel):
@@ -18,22 +16,22 @@
     answer: str
     confidence: float = Field(..., ge=0.0, le=1.0)
     # Breaks in v2: 'min_items' removed -> use 'min_length'
-    sources: List[SourceMatch] = Field(..., min_items=1)
+    sources: List[SourceMatch] = Field(..., min_length=1)
     verified: bool = True

-    class Config:
-        anystr_strip_whitespace = True
+    model_config = ConfigDict(str_strip_whitespace=True)

     # Breaks in v2: @validator(pre=True) -> use @field_validator(mode="before") + @classmethod
-    @validator("query", pre=True)
+    @field_validator("query", mode="before")
+    @classmethod
     def ensure_question(cls, v: str) -> str:
         if not isinstance(v, str) or not v.strip().endswith("?"):
             raise ValueError("Query must end with '?'")
         return v.strip()

     # Breaks in v2: @root_validator -> use @model_validator(mode="after")
-    @root_validator
-    def validate_verification_threshold(cls, values: dict) -> dict:
-        if values.get("confidence", 0.0) < 0.6 and values.get("verified", False):
+    @model_validator(mode="after")
+    def validate_verification_threshold(self) -> "AIQueryResult":
+        if self.confidence < 0.6 and self.verified:
             raise ValueError("Low-confidence answers cannot be marked verified")
```

```
-         return values
+         return self
```